MODULE 16

Exception Handling in Distributed APIs

Build resilient APIs by handling errors gracefully across distributed systems.

TEKsystems Global Services

WORKFORCE DEVELOPMENT

TEKsystems Global Services

WORKFORCE DEVELOPMENT

WHY API ERROR HANDLING MATTERS

Poor error handling leaks internal details, confuses consumers, and makes debugging painful.

POOR ERROR HANDLING

- Inconsistent error formats across endpoints
- Stack traces and internal details exposed
- Generic "500 error" with no useful information
- Hard for consumers to handle programmatically

VS

GOOD ERROR HANDLING

- Consistent, structured error responses
- No sensitive internal details exposed
- Meaningful codes, messages, and correlation IDs
- Easy for consumers to parse and act on

MORE REASONS IT MATTERS

- Improves reliability — graceful handling prevents crashes and cascading failures.
- Enhances user experience — clear, consistent errors show clients what to fix.
- Simplifies client integration and speeds up troubleshooting.
- Builds trust and professionalism: "Errors are not failures — they are opportunities to communicate, guide, and recover."

KEY TAKEAWAY Good error handling is part of your API contract — it builds trust and makes integration easier for consumers.

API ERROR HANDLING

1. REQUEST FLOW

Client

HTTP Request

API

HTTP Response

2. ERROR SOURCES

- Network / Connectivity Failures
- Timeouts
- Downstream Service Failures
- Business Rule Violations
- Validation Errors
- Security / Authorization Failures
- Unexpected Exceptions

3. GLOBAL EXCEPTION HANDLING

@ControllerAdvice

Exception Thrown in Controller / Service

↓

Exception Interceptor

↓

Exception Handler Methods

↓

Build Error Response

↓

Return Response to Client

4. ERROR RESPONSE MODEL

```
{
    "timestamp": "2026-07-17T10:15:30Z",
    "status": 400,
    "error": "Bad Request",
    "message": "Invalid input provided",
    "path": "/api/orders",
    "errorCode": "VALIDATION_ERROR",
    "details": [
      {
          "field": "email",
          "message": "Email is invalid"
      }
    ]
}
```

5. HTTP STATUS CODES

400	Bad Request	Client error (validation, parsing)
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource not found
409	Conflict	Business rule violation
422	Unprocessable Entity	Validation failed
500	Internal Server Error	Unexpected server error
503	Service Unavailable	Dependency/service down

6. LOGGING & MONITORING

- Log Error Details
- Correlate (Request ID)
- Alert Team
- Monitor & Analyze

7. BEST PRACTICES

- Use consistent error response structure
- Do not expose sensitive internal information
- Use meaningful error codes and messages
- Log with context (Request ID, User, Time, Path)
- Handle and mask security errors properly
- Monitor, alert and improve continuously

DISTRIBUTED SYSTEM FAILURES

In distributed systems, failures are the norm, not the exception — design for them.

Network Failures

Timeouts, dropped connections, DNS failures.

Latency & Timeouts

Slow downstream services blocking requests.

Service Unavailability

A dependency is down or overloaded.

Partial Failures

Some operations succeed, others fail.

Retries & Duplicates

Retries can cause duplicate side effects.

Cascading Failures

One failure triggers failures elsewhere.

Exception handling communicates failure. Resilience patterns (retry, timeout, circuit breaker, fallback) attempt recovery or containment — covered in detail later.

DOWNSTREAM SITUATION	API BEHAVIOR
Connection failure	502 or 503, depending on context
Timeout	504 when acting as gateway/proxy
Circuit open	Usually 503
Rate limited downstream	Propagate or map 429 carefully
Partial success	Domain-specific result, compensation, or async status
Unknown outcome after timeout	Do not blindly retry non-idempotent operations

KEY TAKEAWAY Design for failure: timeouts, retries with backoff, circuit breakers, and idempotency are essential in distributed APIs.

DISTRIBUTED SYSTEM FAILURES

FAILURE SOURCES

Network
Latency, packet loss, partitions

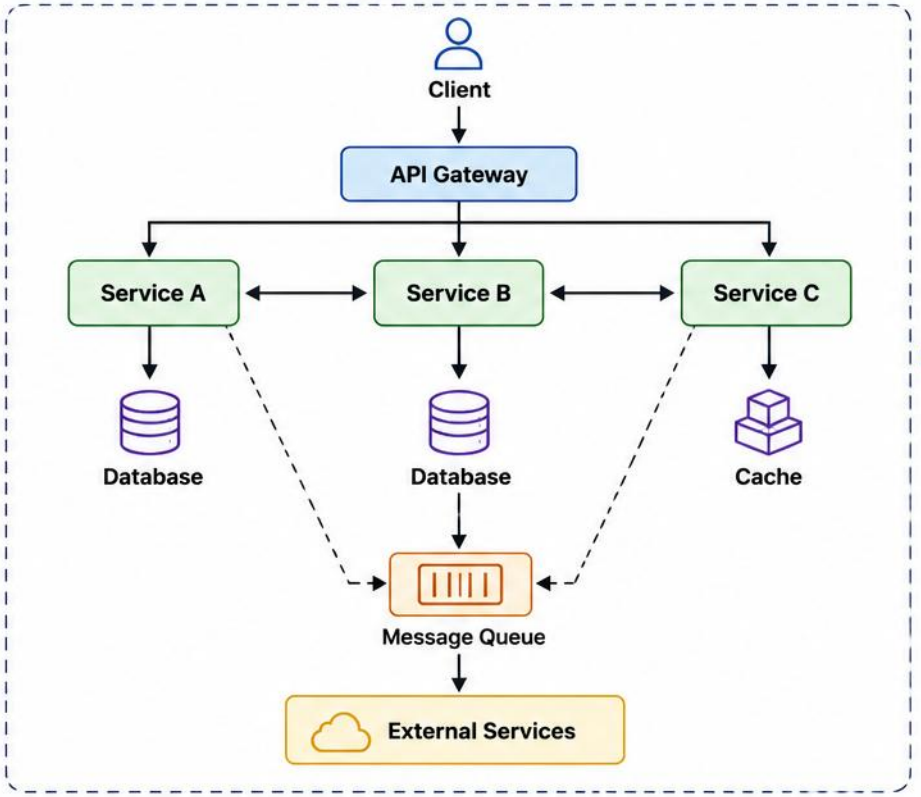
Services
Crashes, overload, bugs

Data Stores
Unavailability, corruption, slow responses

Caches / Queues
Eviction, overflow, message loss

Timeouts
Slow responses, exceeded limits

Security
Auth failures, token expiry, authorization



IMPACTS

High Latency
Slow responses, timeouts

Partial Failures
Some services unavailable

Cascading Failures
Failures spread across dependent services

Data Inconsistency
Stale, lost or corrupted data

Poor User Experience
Errors, unavailability, degraded performance

Business Impact
Revenue loss, SLA breaches

COMMON DISTRIBUTED SYSTEM FAILURES

NETWORK FAILURES
- Packet loss
- High latency
- Network partition

SERVICE FAILURES
- Service crash
- Overloaded
- Deadlock

TIMEOUTS
- Connection timeout
- Read/Write timeout
- Upstream timeout

DEPENDENCY FAILURES
- Downstream service unavailable
- Third-party API down

DATA FAILURES
- DB unavailable
- Data corruption
- Replication lag

RESOURCE FAILURES
- CPU / Memory exhaustion
- Disk full

SECURITY FAILURES
- Auth service down
- Token expiration
- Authorization errors

MESSAGE FAILURES
- Message loss
- Queue overflow
- Duplicate messages

ERROR CLASSIFICATION

Classify errors by type so each can be handled, logged, and reported appropriately.

CLASS	MEANING	TYPICAL STATUS
`Client Input`	Malformed, invalid, or missing data	400
`Domain Conflict`	Business rule or state violation	409
`Authentication / Authorization`	Identity or permission failure	401 / 403
`Resource Lookup`	Requested resource absent	404
`Dependency Failure`	Timeout, unavailable downstream, bad gateway	502 / 503 / 504
`Internal Failure`	Unexpected application defect	500

CLASSIFICATION DIMENSIONS

- Source: client, domain, dependency, internal. Retryability: retryable, non-retryable, unknown. Exposure: consumer-safe detail vs. internal-only detail.

ERROR	SOURCE	RETRYABLE?	CLIENT DETAIL
Invalid email	Client	No	Field-level detail
Duplicate customer	Domain	No	Stable conflict code
Downstream timeout	Dependency	Maybe	Generic temporary-failure message
Null pointer	Internal	No	Generic server error

KEY TAKEAWAY A clear error classification scheme is the foundation for consistent handling, logging, and HTTP status mapping.

CLASSIFYING ERRORS IN CODE

Represent each error class as a distinct Java type so classification drives handling automatically.

EXAMPLE — CLASSIFIED EXCEPTION HIERARCHY

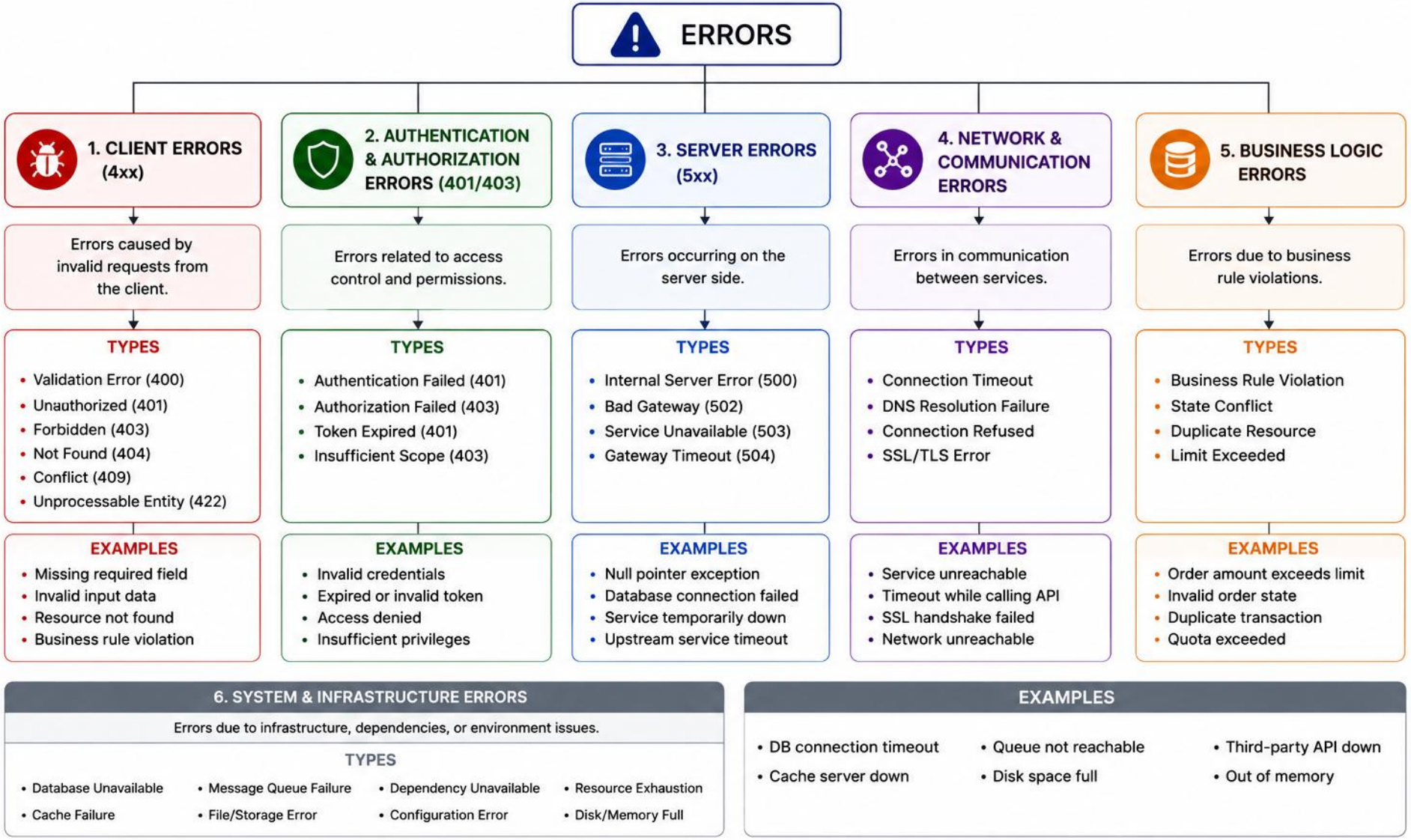
```
public enum ErrorClass {
    CLIENT_INPUT, DOMAIN_CONFLICT, AUTHENTICATION,
    DEPENDENCY, INTERNAL
}
public abstract class ApiException
        extends RuntimeException {
    private final ErrorClass errorClass;
    private final int status;
    protected ApiException(ErrorClass ec,
            int status, String msg) {
        super(msg);
        this.errorClass = ec;
        this.status = status;
    }
}
```

KEY CONCEPTS

- The error class drives both the HTTP status and the log level — decide it once, at the throw site.
- A shared abstract base keeps every subclass consistent: same fields, same constructor contract.
- Concrete exceptions extend ApiException and pass their own ErrorClass to the superclass.

KEY TAKEAWAY Classifying errors as a Java type lets the compiler and the global handler agree on how each failure should be treated.

ERROR CLASSIFICATION



GLOBAL REST EXCEPTION HANDLING WITH @RESTCONTROLLERADVICE

Centralize exception handling so every controller returns consistent, well-formed error responses.

BENEFITS

Cleaner Controllers

No repeated try/catch blocks.

Consistent Responses

Same format everywhere.

Easier Maintenance

One place to update error logic.

```
@ExceptionHandler(CustomerNotFoundException.class) ...
@ExceptionHandler(MethodArgumentNotValidException.class) ...
@ExceptionHandler(Exception.class) // fallback, last
```

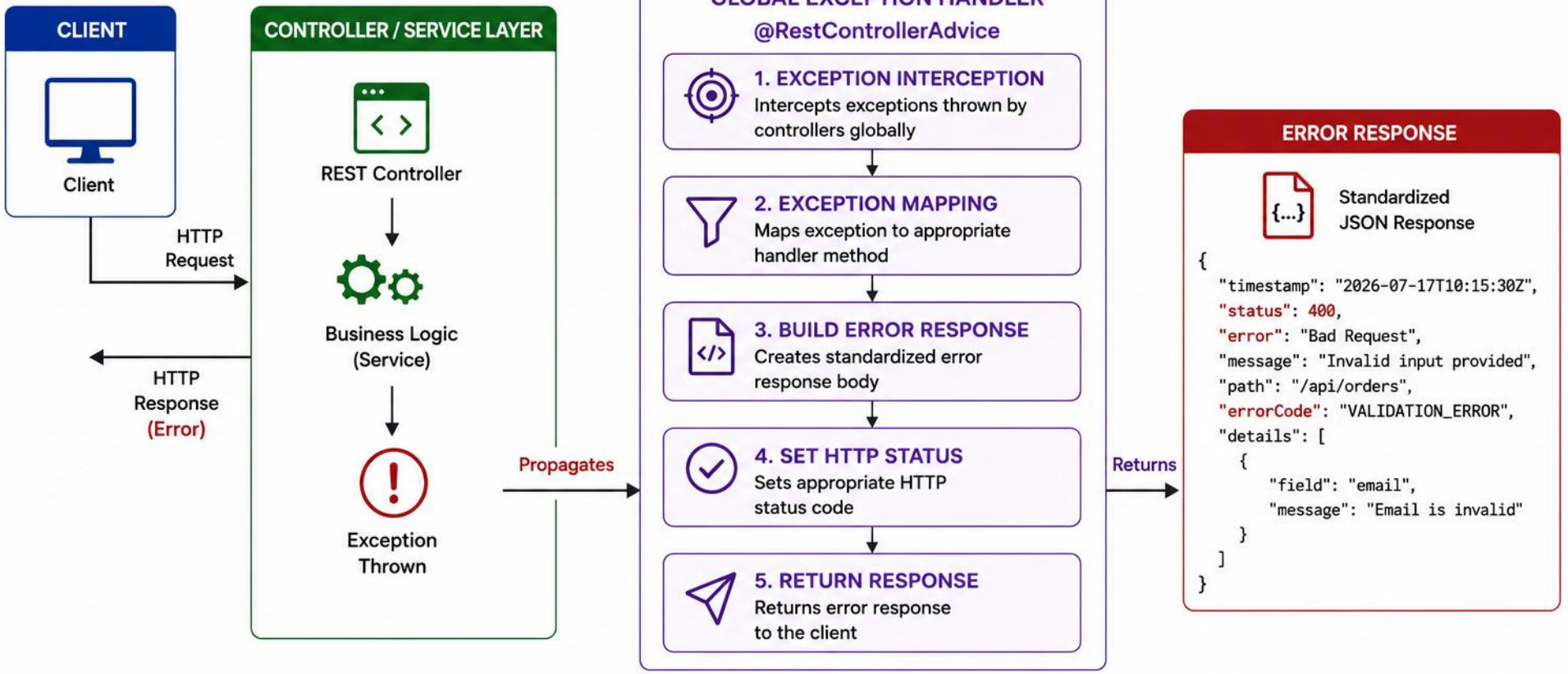
HANDLER ORDER & SCOPE

Specific handlers first, framework rules next, generic fallback last. Never leak raw exception text; log unexpected failures with a trace/correlation ID.

@ControllerAdvice centralizes cross-controller advice; @RestControllerAdvice combines it with response-body serialization.

KEY TAKEAWAY Global exception handling removes duplicated error-handling code and guarantees consistency across every endpoint.

GLOBAL EXCEPTION HANDLING



HANDLED EXCEPTIONS (EXAMPLES)

ValidationException	ResourceNotFoundException	UnauthorizedException	BusinessException	ExternalServiceException	Exception (Generic)
@ExceptionHandler	@ExceptionHandler	@ExceptionHandler	@ExceptionHandler	@ExceptionHandler	@ExceptionHandler
400 Bad Request	404 Not Found	401 Unauthorized	422 Unprocessable Entity	503 Service Unavailable	500 Internal Server Error

TRY IT YOURSELF — EXERCISE 1: CATCH ORDER

Reinforces: the handler order & scope you just saw — specific before generic.

STEP	WHAT TO DO
Goal	Order catch/handler blocks from the most specific domain exception down to generic Exception
File	notes/lab16-catch-order.md (in examples/module-16-exercises/)
Types	NotFoundException, ConflictException, ValidationException, Exception
Order	1) NotFoundException 2) ConflictException 3) ValidationException 4) Exception (fallback, last)
Why	One sentence: a broad catch first would shadow the specific domain mapping below it
Reference	exercises/exercise-01-catch-order.md

```
$ cat handler-order-notes.md
@ExceptionHandler(NotFoundException.class)   // most specific first
@ExceptionHandler(ConflictException.class)
@ExceptionHandler(ValidationException.class)
@ExceptionHandler(Exception.class)                // fallback, last -- pre-lab notes only
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-catch-order.md.

ERROR RESPONSE MODELS

A standardized error response model ensures every error looks the same, no matter which endpoint failed.

STANDARD ERROR RESPONSE — RFC 9457 PROBLEM DETAILS

```
{
  "type": "https://api.example.com/problems/validation",
  "title": "Validation failed",
  "status": 400,
  "detail": "One or more fields are invalid.",
  "instance": "/api/users",
  "code": "VALIDATION_FAILED",
  "correlationId": "3f9c2a6e7b4dd4e1a",
  "errors": [
    { "field": "email", "code": "EMAIL_INVALID",
      "message": "Enter a valid email address." }
  ]
}
```

KEY CONCEPTS

- Correlation ID vs. trace ID: correlationId is an application-level ID that links related logs/requests. A trace ID is a distributed-tracing ID for one trace. They may match in a simple app, but don't assume they're identical — observability platforms may add their own trace and span IDs.
- Timestamp is optional per your organization's standard, not a mandatory field.
- Design principles (from user-friendly error design): be clear and specific; include actionable detail (what field, what rule); link documentation by error type; keep tone professional and consistent across the API.

KEY TAKEAWAY A consistent error model — timestamp, status, error code, message, path, trace ID — makes APIs predictable and debuggable.

ERROR RESPONSE MODELS IN JAVA

Represent the standard error shape as an immutable Java record instead of hand-built JSON.

EXAMPLE — ERRORRESPONSE AS A JAVA RECORD

```
public record ErrorResponse(
        String type,
        String title,
        int status,
        String detail,
        String instance,
        String code,
        String correlationId,
        List<FieldError> errors) {

    public record FieldError(
            String field, String message) {}
}
```

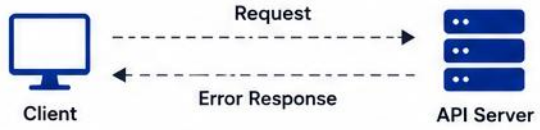
KEY CONCEPTS

- A record is a natural fit for an error response: immutable, concise, with built-in equals and toString.
- Nesting FieldError as an inner record keeps validation detail scoped to where it is used.
- Serializers such as Jackson map record components to JSON fields automatically — no boilerplate getters.

KEY TAKEAWAY Modeling the error response as a record keeps the JSON contract and the code in lockstep — no drift between them.

ERROR RESPONSE MODELS

1. SIMPLE ERROR MODEL

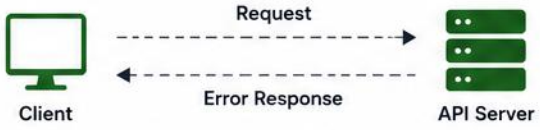


JSON RESPONSE EXAMPLE

```
{
  "timestamp": "2026-07-17T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Invalid input provided"
}
```

USE CASE: Simple APIs, basic error information

2. DETAILED ERROR MODEL

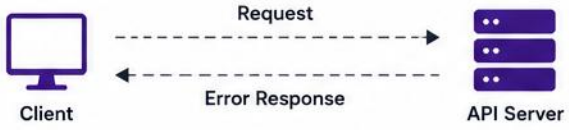


JSON RESPONSE EXAMPLE

```
{
  "timestamp": "2026-07-17T10:15:30Z",
  "status": 422,
  "error": "Unprocessable Entity",
  "message": "Validation failed",
  "path": "/api/users",
  "errorCode": "VALIDATION_ERROR",
  "details": [
    { "field": "email", "message": "Email is invalid" },
    { "field": "age", "message": "Age must be >= 18" }
  ]
}
```

USE CASE: Validation errors, detailed field-level info

3. PROBLEM DETAILS (RFC 7807) MODEL

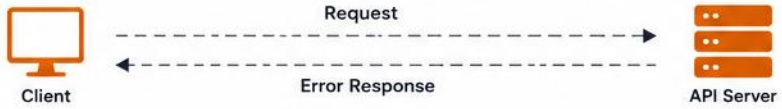


JSON RESPONSE EXAMPLE

```
{
  "type": "https://example.com/probs/invalid-input",
  "title": "Invalid Input",
  "status": 400,
  "detail": "The request contains invalid input",
  "instance": "/api/users/123",
  "extensions": {
    "errorCode": "INVALID_INPUT",
    "timestamp": "2026-07-17T10:15:30Z",
    "path": "/api/users/123"
  }
}
```

USE CASE: Standardized error format (RFC 7807)

4. APPLICATION SPECIFIC ERROR MODEL

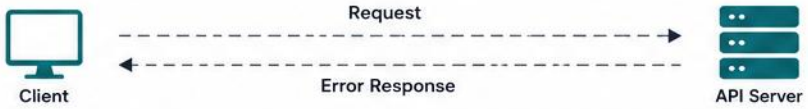


JSON RESPONSE EXAMPLE

```
{
  "success": false,
  "data": null,
  "error": {
    "code": "USR_1001",
    "message": "User not found",
    "description": "No user exists with the given ID",
    "timestamp": "2026-07-17T10:15:30Z",
    "requestId": "a8f3d2e1-7b4c-4d2a-bc9f-123456789abc"
  }
}
```

USE CASE: Domain-driven applications, consistent envelope

5. ERROR MODEL WITH TRACE & SUPPORT INFO



JSON RESPONSE EXAMPLE

```
{
  "timestamp": "2026-07-17T10:15:30Z",
  "status": 500,
  "error": "Internal Server Error",
  "message": "An unexpected error occurred",
  "path": "/api/orders",
  "traceId": "4f3a1b2c9d4e7f8a",
  "support": {
    "contact": "support@example.com",
    "documentation": "https://docs.example.com/errors/500"
  }
}
```

USE CASE: Production APIs, easier debugging & support

TRY IT YOURSELF — EXERCISE 2: ERRORRESPONSE JSON DRAFT

Reinforces: the standard error response fields you just saw.

STEP	WHAT TO DO
Goal	Draft ErrorResponse JSON fields for a not-found error, including correlation
File	notes/lab16-errorresponse-draft.md (in examples/module-16-exercises/)
Fields	timestamp, status, error, message, path, correlationId
Fixture	CUS-9999 not found, correlationId lab-request-001
Hygiene	Message must never include stack traces or raw SQL
Reference	exercises/exercise-02-errorresponse-json.md

```
$ cat errorresponse-draft.json
{ "status": 404, "error": "NOT_FOUND",
  "message": "Customer CUS-9999 was not found.",  // safe, no SQL/stack trace
  "correlationId": "lab-request-001" }        // paper draft only
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-errorresponse-json.md.

HTTP STATUS CODES

Map exceptions to the right HTTP status code — it's part of your API contract.

CODE	MEANING	WHEN TO USE
400	Bad Request	Validation failures, malformed input
401	Unauthorized	Missing or invalid authentication
403	Forbidden	Authenticated but not authorized
404	Not Found	Resource doesn't exist
409	Conflict	Duplicate resource, state conflict
500	Internal Server Error	Unexpected server-side failure
503	Service Unavailable	Downstream dependency unavailable
422	Unprocessable Entity	Semantically invalid request under a standard that distinguishes it from 400
429	Too Many Requests	Client exceeded rate limit
502	Bad Gateway	Invalid response from downstream service
504	Gateway Timeout	Downstream timeout while acting as gateway/proxy

Follow the organization's API standard consistently. 400 is common; some APIs use 422 for semantic validation failures.

KEY TAKEAWAY Choosing the correct HTTP status code helps consumers handle errors programmatically and correctly.

MAPPING EXCEPTIONS TO STATUS CODES

Centralize the exception-to-status mapping in one method so every handler stays consistent.

EXAMPLE — CENTRALIZED STATUS MAPPING

```
int statusFor(ApiException ex) {
    return switch (ex.errorClass()) {
        case CLIENT_INPUT -> 400;
        case AUTHENTICATION -> 401;
        case DOMAIN_CONFLICT -> 409;
        case DEPENDENCY -> 503;
        case INTERNAL -> 500;
    };
}
ResponseEntity<ErrorResponse> handle(
        ApiException ex) {
    int status = statusFor(ex);
    return ResponseEntity.status(status)
            .body(ErrorResponse.from(ex, status));
}
```

KEY CONCEPTS

- A switch expression centralizes the mapping in one place — no magic numbers scattered across handlers.
- Compute the status once, then reuse it for both the response and the log level.
- ResponseEntity.status(...).body(...) keeps the controller in full control of the wire response.

KEY TAKEAWAY Deriving status codes from the error class, not the exception name, keeps the mapping stable as the hierarchy grows.

HTTP STATUS CODES

CLASS	RANGE	DESCRIPTION	COMMON CODES		MEANING / WHEN TO USE
1xx Informational	100 – 199	Request received, continuing process	100	Continue	Request received, client should continue
			101	Switching Protocols	Switching to a different protocol
			102	Processing	Processing request, no response yet
2xx Success	200 – 299	Request successfully received, understood, and accepted	200	OK	Request successful
			201	Created	Resource created successfully
			204	No Content	Request successful, no content to return
			206	Partial Content	Partial content returned
3xx Redirection	300 – 399	Further action needs to be taken by the client	301	Moved Permanently	Resource permanently moved
			302	Found	Resource temporarily moved
			304	Not Modified	Resource not modified, use cached version
			307	Temporary Redirect	Temporary redirect, repeat request with new URL
			308	Permanent Redirect	Permanent redirect, repeat request with new URL
4xx Client Error	400 – 499	Request contains bad syntax or cannot be fulfilled	400	Bad Request	Invalid request syntax
			401	Unauthorized	Authentication required or failed
			403	Forbidden	Client doesn't have permission
			404	Not Found	Requested resource not found
			409	Conflict	Request conflicts with current state
			422	Unprocessable Entity	Request well-formed but cannot be processed
5xx Server Error	500 – 599	Server failed to fulfill a valid request	500	Internal Server Error	Unexpected server error
			501	Not Implemented	Server does not support the functionality
			502	Bad Gateway	Invalid response from upstream server
			503	Service Unavailable	Server is currently unavailable
			504	Gateway Timeout	Gateway did not receive timely response

TRY IT YOURSELF — EXERCISE 3: FAILURE TO STATUS MAP

Reinforces: the HTTP status code table you just saw.

STEP	WHAT TO DO
Goal	Map Northstar failures to client-facing status classes
File	notes/lab16-status-map.md (in examples/module-16-exercises/)
Not found	CUS-9999 not found → 404 / SOAP Client fault
Conflict	Activate Amina illegal transition → 409 or 422 (pick one, write why)
Never	Never return 200 with an error payload for any of these failures
Reference	exercises/exercise-03-failure-status-map.md

```
$ cat lab16-status-map.md
CUS-9999 not found                    -> 404 / SOAP Client fault
Activate Amina illegal transition      -> 409 or 422
Validation blank name                  -> 400
Unexpected bug                         -> 500 (generic message)   // never 200
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-failure-status-map.md.

BUSINESS EXCEPTIONS

Business exceptions represent violations of domain rules — invalid operations, entity state issues, duplicates.

EXAMPLE

```
public abstract class BusinessException
        extends RuntimeException {

    private final String code;
    private final int status;
}
```

BEST PRACTICES

- Map status by meaning, not exception class name (see table below).
- Use specific exception types only where callers need distinct handling.
- A shared typed exception with a stable error code can suit related failures.
- Never expose internal state, stack traces, or raw exception messages to consumers.

STATUS MAPPING BY MEANING

MEANING	TYPICAL STATUS
Duplicate resource	409
Invalid state transition	Often 409
Semantically invalid command	422 or 400
Insufficient funds	Domain-specific — often 409 or 422
Business authorization failure	May be 403

USAGE EXAMPLE

```
throw new BusinessException(
    CustomerErrorCode.INVALID_STATUS_TRANSITION,
    "Customer cannot move from ACTIVE to PROSPECT");
```

Consumer-facing messages should not come directly from raw exception messages. The exception carries a stable internal code and safe parameters; the handler maps it to a consumer-safe message; logs retain the technical details; the response follows the documented error contract.

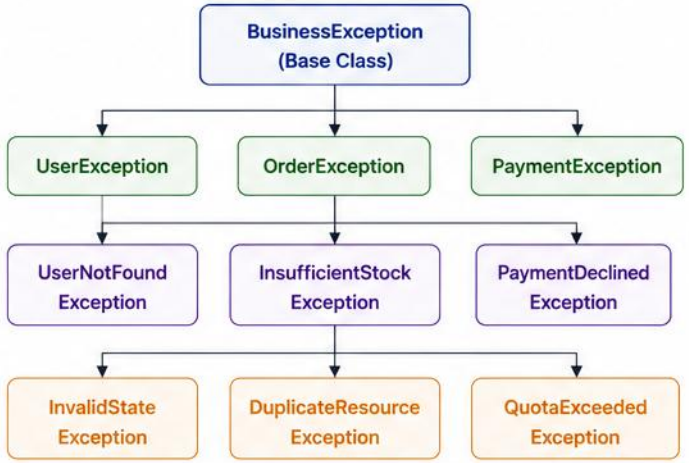
KEY TAKEAWAY Well-named business exceptions make your code self-documenting and your error responses meaningful.

BUSINESS EXCEPTIONS

COMMON BUSINESS EXAMPLES

SCENARIO	EXAMPLE
User related	User not found, user already exists, account locked
Order related	Insufficient stock, invalid order state, order cannot be canceled
Payment related	Payment declined, invalid payment method, limit exceeded
Resource related	Resource not available, quota exceeded, duplicate resource
Authorization related	Operation not allowed, invalid role for operation

BUSINESS EXCEPTION HIERARCHY



BEST PRACTICES

- Use meaningful exception names that reflect business rules
- Include clear, user-friendly messages
- Include error codes for consistent handling
- Do not expose internal details
- Log exception with full context for troubleshooting

EXAMPLE BUSINESS EXCEPTION RESPONSE

```
{
  "timestamp": "2026-07-17T10:15:30Z",
  "status": 400,
  "error": "Business Rule Violation",
  "message": "Insufficient stock for product with id 12345",
  "errorCode": "ORDER_STOCK_INSUFFICIENT",
  "path": "/api/orders",
  "details": { "productId": "12345", "requestedQty": 10, "availableQty" : 2 }
}
```

FIELD	DESCRIPTION
timestamp	Time when the error occurred
status	HTTP status code
error	Short error summary
message	Human-readable error message
errorCode	Application-specific error code
path	API endpoint where error occurred
details	Additional context for the error

TRY IT YOURSELF — EXERCISE 4: MESSAGE HYGIENE (STARTER)

Reinforces: never expose raw exception messages or stack traces to consumers.

STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK STARTER — complete the safe-vs-unsafe message TODOs (not from scratch)
File	notes/lab16-hygiene-todos.md (in examples/module-16-exercises/)
Fill in	6 blanks: safe message, unsafe anti-pattern, correlation field, log/return-to-client?, live-advice?
Server vs. client	Log stack trace on the server: yes. Return stack trace to the client: no.
Correlation	Every error sketch includes lab-request-001 (or the incoming request header value)
Reference	exercises/exercise-04-fill-message-hygiene-todos.md (starter)

```
$ cat lab16-hygiene-todos.md
Safe not-found message: _____
Unsafe message anti-pattern: _____
Correlation always field: _____              // correlationId
Log stack trace? _____   Return to client? _____ // yes (server) / no (client)
```

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-fill-message-hygiene-todos.md (starter).

VALIDATION EXCEPTIONS

Module 14 covered Bean Validation in depth — here we map validation exceptions to a stable field-error response.

EXAMPLE HANDLER

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<ErrorResponse> handleValidation(
    MethodArgumentNotValidException ex) {...}
```

EXAMPLE FIELD ERRORS (NESTED / INDEXED PATHS)

```
"errors": [
  { "field": "address.postalCode",
    "code": "FIELD_INVALID",
    "message": "Enter a valid postal code" },
  { "field": "items[2].quantity",
    "code": "FIELD_INVALID",
    "message": "Quantity must be positive" }
]
```

BEST PRACTICES

- Mapping pipeline: framework exception → field extraction → stable field-error model → status selection → correlation ID.
- Preserve a stable, consumer-understandable field path for nested and collection validation errors (see example, right) — not just a flat "field": "email".
- Follow the organization's API standard consistently. 400 is common; some APIs use 422 for semantic validation failures.

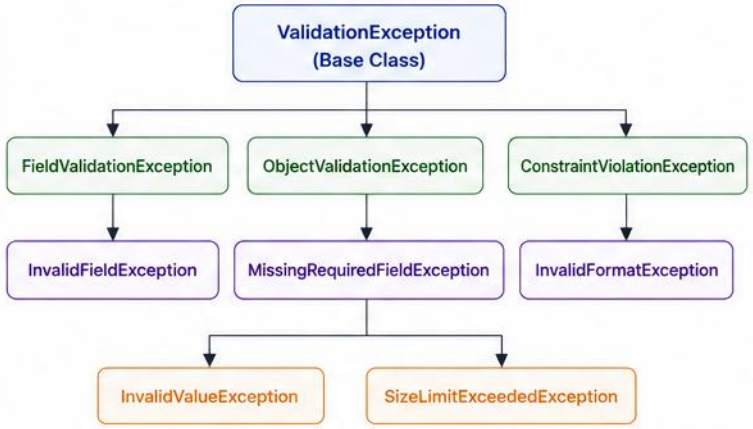
KEY TAKEAWAY Field-level validation errors help API consumers fix their requests quickly, without guesswork.

VALIDATION EXCEPTIONS

COMMON VALIDATION SCENARIOS

SCENARIO	EXAMPLES
Required Field Missing	name, email, address
Invalid Data Type	Expected number but received string
Length/Size Constraint	Minimum/maximum length violated
Format Constraint	Invalid email, phone number, date format
Range Constraint	Value out of allowed range (min/max)
Uniqueness Constraint	Duplicate value for unique field

VALIDATION EXCEPTION HIERARCHY



BEST PRACTICES

- Validate input at the boundary (API layer)
- Return clear and specific validation messages
- Provide field-level error details
- Use consistent error codes for validation errors
- Do not expose internal validation rules

EXAMPLE VALIDATION ERROR RESPONSE

```
{
  "timestamp": "2026-07-17T10:15:30Z",
  "status": 400,
  "error": "Validation Failed",
  "message": "Request data validation failed",
  "path": "/api/users",
  "errorCode": "VALIDATION_ERROR",
  "validationErrors": [
    { "field": "email", "message": "Invalid email format", "rejectedValue": "john@com" },
    { "field": "age", "message": "Must be between 18 and 65", "rejectedValue": 15 },
    { "field": "name", "message": "Name is required", "rejectedValue": null }
  ]
}
```

FIELD	DESCRIPTION
timestamp	Time when the error occurred
status	HTTP status code (400 Bad Request)
error	Short error summary
message	Human-readable error message
path	API endpoint where error occurred
errorCode	Application-specific error code
validationErrors	List of field-level validation errors
field	Name of the field that failed validation
message	Validation error message for the field
rejectedValue	Value that was rejected

SECURITY EXCEPTIONS

Authentication and authorization failures need careful handling — informative, but not revealing.

EXCEPTION	MEANING	STATUS CODE
`AuthenticationException`	Missing or invalid credentials	401 Unauthorized
`AccessDeniedException`	Authenticated but lacks permission	403 Forbidden
`TokenExpiredException`	JWT/session token has expired	401 Unauthorized

BEST PRACTICES

- Never reveal whether a username or resource exists in error messages.
- Log security exceptions with context, but don't return internal details to the client.
- Return generic, consistent messages for authentication failures.
- Depending on the threat model, an API may intentionally return the same response for nonexistent and unauthorized resources (e.g., 404 instead of 403) — follow the security standard consistently.
- Expose only the minimum detail required by the client contract. OAuth-based APIs may use standardized authentication error fields while avoiding sensitive internals.

KEY TAKEAWAY Security exceptions must be handled carefully — informative to developers via logs, generic to the client.

HANDLING SECURITY EXCEPTIONS

Authentication and authorization failures need a dedicated handler that stays generic to the client.

EXAMPLE — AUTHENTICATION EXCEPTION HANDLER

```
@ExceptionHandler(AuthenticationException.class)
public ResponseEntity<ErrorResponse> handleAuth(
        AuthenticationException ex) {
    log.warn("Authentication failed: {}",
            ex.getCorrelationId());
    ErrorResponse body = new ErrorResponse(
            401, "Unauthorized",
            "Authentication failed",
            ex.getCorrelationId());

    return ResponseEntity.status(401).body(body);
}
```

BEST PRACTICES

- Log the correlation ID and internal detail; return only a generic, consumer-safe message.
- Use the same handler shape for every AuthenticationException subtype for a consistent 401 response.
- Never include the username, token, or denial reason in the response body.

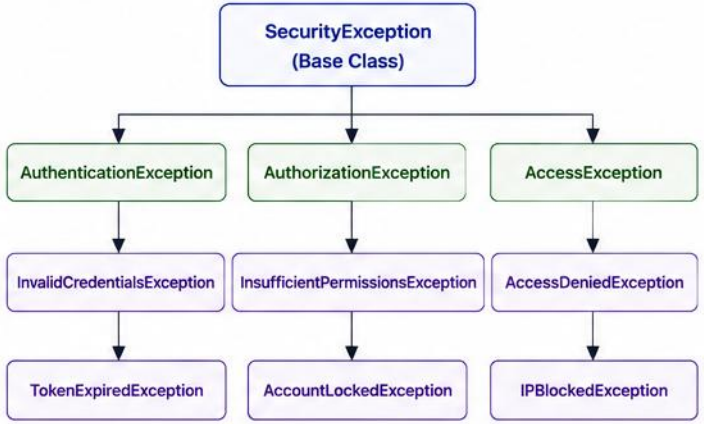
KEY TAKEAWAY A dedicated handler keeps authentication failures informative in the logs and uninformative to attackers.

SECURITY EXCEPTIONS

COMMON SECURITY EXCEPTIONS

EXCEPTION	DESCRIPTION
AuthenticationException	Authentication failed (invalid credentials, token expired)
AuthorizationException	User not authorized to access resource
AccessDeniedException	Access is denied (insufficient permissions)
AccountLockedException	User account is locked
TokenExpiredException	Security token has expired
CSRFException	Invalid or missing CSRF token

SECURITY EXCEPTION HIERARCHY



BEST PRACTICES

- Do not expose sensitive security details
- Use generic messages for security errors
- Log full details for monitoring and analysis
- Use proper HTTP status codes
- Prevent user enumeration in error messages
- Review and audit security logs regularly

EXAMPLE SECURITY ERROR RESPONSE

```
{
  "timestamp": "2026-07-17T10:15:30Z",
  "status": 401,
  "error": "Unauthorized",
  "message": "Authentication failed",
  "errorCode": "AUTHENTICATION_FAILED",
  "path": "/api/orders",
  "traceId": "4fa3a1b2c9d4e7f8a",
  "details": null
}
```

FIELD	DESCRIPTION
timestamp	Time when the error occurred
status	HTTP status code
error	Short error summary
message	Human-readable message
errorCode	Application-specific error code
path	API endpoint where error occurred
traceId	Unique ID for tracing the request
details	Additional details (if safe to expose)

LOGGING ERRORS

Good logging turns a production incident into a quick fix — log with context, not noise.

Correlation IDs

- Trace a request across services and logs.

Right Log Level

- Depends on the situation — see table below.

No Sensitive Data

- Never log passwords, tokens, or PII; use non-sensitive stable IDs.

Structured Logging

- JSON logs for easy parsing and search.

Include Context

- Sufficient diagnostic context, not everything available.

Monitor & Alert

- Set up alerts on error rate spikes.

SITUATION	TYPICAL LOGGING
Client validation error	No error stack trace; metric or debug if needed
Expected domain conflict	Info/debug or structured event
Security failure	Security audit event, carefully rate-limited
Downstream transient failure	Warn/error based on impact
Unexpected internal failure	Error with stack trace

USER IDS & DUPLICATE LOGGING

Use non-sensitive, stable identifiers where approved; avoid emails, tokens, personal data, and raw payloads. Hash or tokenize identifiers if policy requires.

Log once at the boundary that has enough context to act. Lower layers should enrich or rethrow, not repeatedly log the same stack trace.

KEY TAKEAWAY Log sufficient diagnostic context, not all available context — apply redaction, allowlists, and size limits so correlation IDs and structured logs stay fast and traceable.

STRUCTURED ERROR LOGGING

Log with correlation context so a production incident becomes a quick fix, not a search.

EXAMPLE — MDC CORRELATION LOGGING

```
try {
    MDC.put("correlationId", correlationId);
    return service.process(request);
} catch (BusinessException ex) {
    log.warn("Business rule violated: code={}",
            ex.getCode());
    throw ex;
} catch (Exception ex) {
    log.error("Unexpected failure processing {}",
            request.id(), ex);
    throw ex;
} finally {
    MDC.remove("correlationId");
}
```

BEST PRACTICES

- Put the correlation ID in the MDC once per request so every log line in that request carries it.
- Log expected business failures at WARN with the stable error code; reserve ERROR for the unexpected.
- Always clear the MDC in a finally block so IDs never leak into unrelated requests on the same thread.

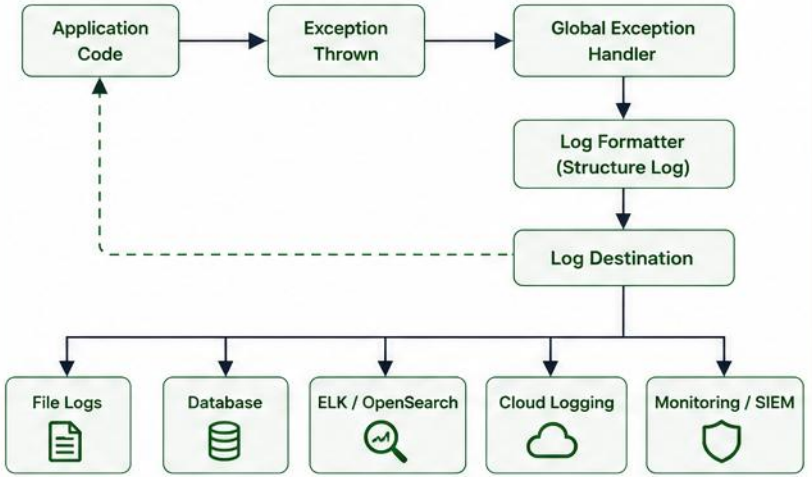
KEY TAKEAWAY MDC-based structured logging turns a scattered stack trace into a searchable, correlated incident timeline.

LOGGING ERRORS

WHAT TO LOG

- Error type / exception class
- Error message
- Stack trace
- User ID / affected user
- Request path & HTTP method
- Correlation / Trace ID
- Request parameters (sanitized)
- Timestamp
- Environment / Service name
- Host / IP address

LOGGING FLOW



BEST PRACTICES

- Log at the right level (ERROR, WARN, INFO)
- Use structured logging (JSON) for easy parsing
- Never log sensitive data (passwords, tokens, PII)
- Include correlation/trace ID for end-to-end tracking
- Centralize logs for analysis and retention
- Set log retention and rotation policies
- Monitor logs and set up alerts for critical errors

EXAMPLE STRUCTURED ERROR LOG (JSON)

```json
{
  "timestamp": "2026-07-17T10:15:30.123Z",
  "level": "ERROR",
  "message": "Failed to process user request",
  "errorType": "ValidationException",
  "stackTrace": "com.example.service.UserService.validate(UserService.java:45)\n....",
  "path": "/api/users",
  "method": "POST",
  "userId": "user123",
  "correlationId": "4fa3a1b2c9d4e7f8a",
  "requestId": "req-7890",
  "ip": "10.0.0.5",
  "service": "user-service",
  "environment": "production"
}
```

LOG LEVELS

LEVEL	DESCRIPTION	WHEN TO USE
ERROR	Serious issues that prevent request completion	Exceptions, failures, application errors
WARN	Potential issues that do not stop execution	Deprecated usage, retries, slow response
INFO	General operational events	Request received, process completed
DEBUG	Detailed diagnostic information	Development and troubleshooting
TRACE	Very detailed execution trace	Deep debugging, performance analysis

ENTERPRISE API ERROR STANDARDS

Consistent error standards across all services make your enterprise APIs predictable and easy to integrate.

Standard Error Schema

- Same fields across every API in the organization.

Error Code Registry

- Type-specific documentation links per code, not one generic URL.

Documented Errors

- Every error code documented in the API contract.

Traceability

- Correlation IDs propagated across services.

Security by Default

- No stack traces or internals in production responses.

Observability

- Metrics and dashboards for error rates by type.

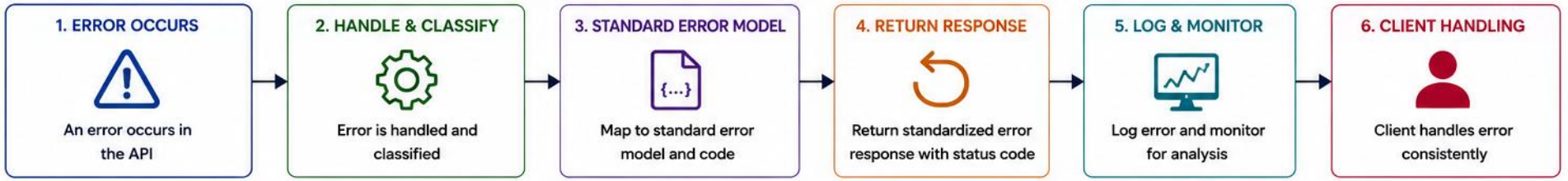
ERROR-CODE REGISTRY GOVERNANCE

Define per code: ownership, naming convention, uniqueness, deprecation, documentation, mapping to HTTP status, retryability, consumer action, and security classification.

CODE	STATUS	RETRYABLE	CONSUMER ACTION
`CUSTOMER_STATUS_CONFLICT`	409	No	Refresh state and choose a valid transition

KEY TAKEAWAY Enterprise error standards make debugging faster and integration easier across every team and service.

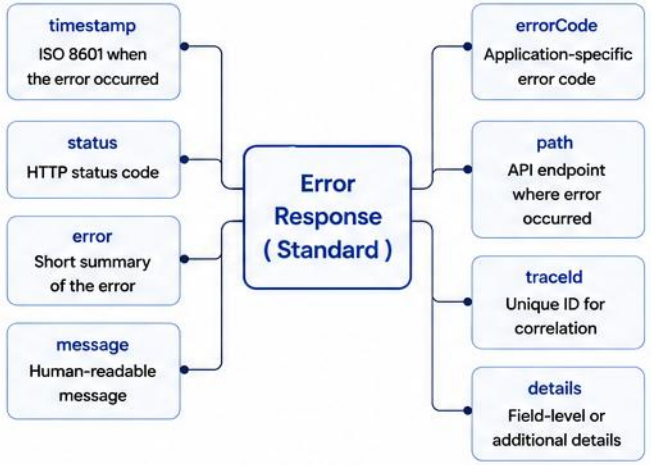
ENTERPRISE API ERROR STANDARDS



ERROR CATEGORIES

CATEGORY	DESCRIPTION	EXAMPLE
VALIDATION	Invalid input or business rule violations	Missing required field, invalid email format
AUTHENTICATION	Authentication failures	Invalid credentials, token expired
AUTHORIZATION	Insufficient permissions	Access denied to resource
BUSINESS	Business rule or process errors	Insufficient balance, order cannot be cancelled
SYSTEM	Server or dependency failures	Database unavailable, service timeout
EXTERNAL	Third-party or external service errors	Payment gateway error, upstream service failure

STANDARD ERROR RESPONSE MODEL



ERROR CODE DESIGN

Structured & Consistent
Use a consistent format and naming convention.

Meaningful
Codes should be descriptive and easy to understand.

Categorized
Group codes by domain and error category.

Unique
Each error condition should have a unique code.

Stable
Once published, codes should remain stable.

EXAMPLE ERROR RESPONSE (JSON)

```
{
  "timestamp": "2026-07-17T10:15:30Z",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for one or more fields.",
  "errorCode": "VAL_1001",
  "path": "/api/users",
  "traceId": "4fa3a1b2c9d4e7f8a1234567890abcdef",
  "details": {
    "fieldErrors": [
      { "field": "email", "rejectedValue": "john@com", "message": "Invalid email format" },
      { "field": "age", "rejectedValue": -5, "message": "Age must be greater than 0" }
    ]
  }
}
```

BEST PRACTICES

- Use appropriate HTTP status codes.
- Provide clear, user-friendly messages.
- Use consistent error response format.
- Maintain a catalog of error codes.
- Do not expose internal implementation details.
- Log errors with full context.
- Monitor and alert on error patterns.
- Review and update standards regularly.
- Document for developers and consumers.
- Ensure security and compliance.

TRY IT YOURSELF — EXERCISE 5: CORRELATION ON EVERY ERROR

Reinforces: correlation IDs propagated across services, on every path.

STEP	WHAT TO DO
Goal	Build a checklist confirming success and failure paths both carry correlation
File	notes/lab16-correlation-always.md (in examples/module-16-exercises/)
Success path	Activate Ravi success still echoes/logs correlationId lab-request-001
Failure path	Not-found CUS-9999 response includes that same correlation field
Missing header	Policy idea: generate a correlation ID when the incoming header is missing
Reference	exercises/exercise-05-correlation-always.md

```
$ cat lab16-correlation-always.md
activateCustomer(Ravi)   success  -> correlationId: lab-request-001
notFound(CUS-9999)       failure  -> correlationId: lab-request-001
header missing           policy   -> generate one  // note for later labs
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-correlation-always.md.

TRY IT YOURSELF — EXERCISE 6: LAB 16 PREP CHECKLIST

Reinforces: status map, JSON draft, catch order, and hygiene TODOs — all four, together, before Lab 16.

STEP	WHAT TO DO
Goal	Confirm exception-handling prep is complete before starting Lab 16 — no live advice controllers yet
File	notes/lab16-prep-checklist.md (in examples/module-16-exercises/)
Artifacts	Confirm the status map, JSON draft, catch order, and hygiene TODOs all exist
Fixtures	Recall CUS-9999 not-found and correlationId lab-request-001
Boundary	Write: pre-lab only — no @ControllerAdvice live yet
Reference	exercises/exercise-06-lab16-prep-checklist.md

```
$ cat lab16-prep-checklist.md
[x] Status map (Ex.1)        [x] ErrorResponse JSON (Ex.2)
[x] Catch order (Ex.3)       [x] Hygiene TODOs (Ex.4)
[x] Correlation always (Exercise 5)   // pass if this note exists, else revisit Ex. 5
```

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-lab16-prep-checklist.md.

LAB OVERVIEW – API EXCEPTION HANDLING

Design a consistent API error model for the Customer Management Platform, then map the pattern to Spring later.

WHAT YOU WILL BUILD

- ExceptionToErrorResponseMapper for validation and business failures (framework-neutral, not Spring's global interception).
- BusinessException factories for not-found and conflict cases.
- Standard ErrorResponse with status mapping and correlation IDs.
- Spring's @RestControllerAdvice delegates to the same mapping policy later.

TECH STACK

ITEM	VALUE
`Language`	Java 21
`Framework`	Java + Bean Validation
`Logging`	stdout / simple logger
`Testing`	JUnit 5 + Maven
`Pattern`	Two-stage: framework-neutral mapper now, Spring adapter preview later

HIGH-LEVEL FLOW

CONVENTION

Domain/service methods may throw typed exceptions; the facade catches and maps them and returns ApiResult. A layer should not both return a failure value AND throw for the same expected condition.

KEY TAKEAWAY Expected outcome: every failure path returns the same safe JSON shape with the right status and correlation ID.

LAB TASKS AND DELIVERABLES

Implement the lab's plain-Java error model, facade mapping, demos, and tests.

#	TASK	DETAILS	FORMAT
1	ExceptionToErrorResponseMapper	Map validation/business/unexpected failures (framework-neutral mapper).	.java
2	BusinessException	Add not-found and conflict factory methods.	.java
3	Standard Error Model	Include status, message, errors, and correlationId.	.java
4	Facade Integration	Return ApiResult.Ok/Fail consistently — don't also throw for the same expected condition.	.java
5	CLI Error Demos	Demonstrate 400, 404, and 409 JSON safely.	.java
6	Tests and Docs	Add JUnit 5 handler tests and document mappings.	Tests / docs

SUCCESS CRITERIA

- All failure paths share one JSON shape; 400/404/409 mappings are tested; correlationId is present; no stack traces or entity details leak.
- 500 unexpected-failure test: generic safe message, correlationId present, no stack trace in the JSON response, stack trace retained in the internal log.
- Field validation tests: stable field path, stable error code, safe message, multiple violations, deterministic ordering (if required), no entity-serialization leakage.
- Correlation ID: accept an approved incoming header or generate one; validate length/characters; include it in logs and response; propagate downstream; never trust it for identity or authorization.
- Serialize errors with the project's JSON serializer — never hand-concatenate untrusted strings into JSON.

KEY TAKEAWAY A robust, secure, well-documented exception handling strategy makes your API easy to integrate and trust.

API ERROR HANDLING — DEFINITION OF DONE

Use this checklist to confirm an exception handling strategy is complete, consistent, and safe before calling it done.

1. Errors are classified by source and retryability.
2. Every public error has a stable code.
3. HTTP status matches the API standard.
4. Response shape follows one documented schema.
5. No stack trace or sensitive data is exposed.
6. Validation errors contain safe field paths.
7. Dependency failures are mapped consistently.
8. Correlation ID is propagated end-to-end.
9. Unexpected failures are logged once with diagnostics.
10. Expected client errors do not create noisy error logs.
11. Error codes are documented and tested.
12. Generic fallback behavior is defined.

KEY TAKEAWAY Reliable APIs handle failure gracefully — consistent, secure, and transparent error handling builds consumer trust.

KNOWLEDGE CHECK

Test your understanding of exception handling in distributed APIs.

1. A downstream inventory service times out while your API acts as a gateway. Likely response?

A. 200 OK, and retry silently

B. 504 Gateway Timeout (architecture/policy-dependent)

C. 401 Unauthorized

D. 201 Created

2. Which status code fits a validation failure?

A. 200 OK

B. 400 Bad Request

C. 404 Not Found

D. 500 Internal Server Error

3. What should an error response NEVER include?

A. A timestamp

B. A trace ID

C. Internal stack traces

D. An error message

4. Which error should normally avoid an ERROR-level stack trace?

A. An unexpected null pointer exception

B. A downstream service crash

C. A normal client validation failure

D. A corrupted database record

KEY TAKEAWAY Map failures to status by meaning, centralize handling, and never expose stack traces to API consumers.